A.Archishman
192425341
ITA0502
COMPUTER VISION
SLOT C

1.Feature Descriptor Comparison (SIFT vs Harris) Design and perform an experiment to compare Harris Corner Detection and SIFT feature extraction on the same set of images. Explain the theoretical principles behind corner detection and feature descriptors. Implement both techniques using suitable software/tools, document the detected features systematically, and analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

A) Aim

To compare Harris Corner Detection and SIFT Feature Detection on the same image.

Algorithm

1. Read the image.
2. Convert it to grayscale.
3. Apply Harris Corner Detection.
4. Apply SIFT Feature Detection.
5. Display both outputs.
6. Compare the detected features.

Procedure

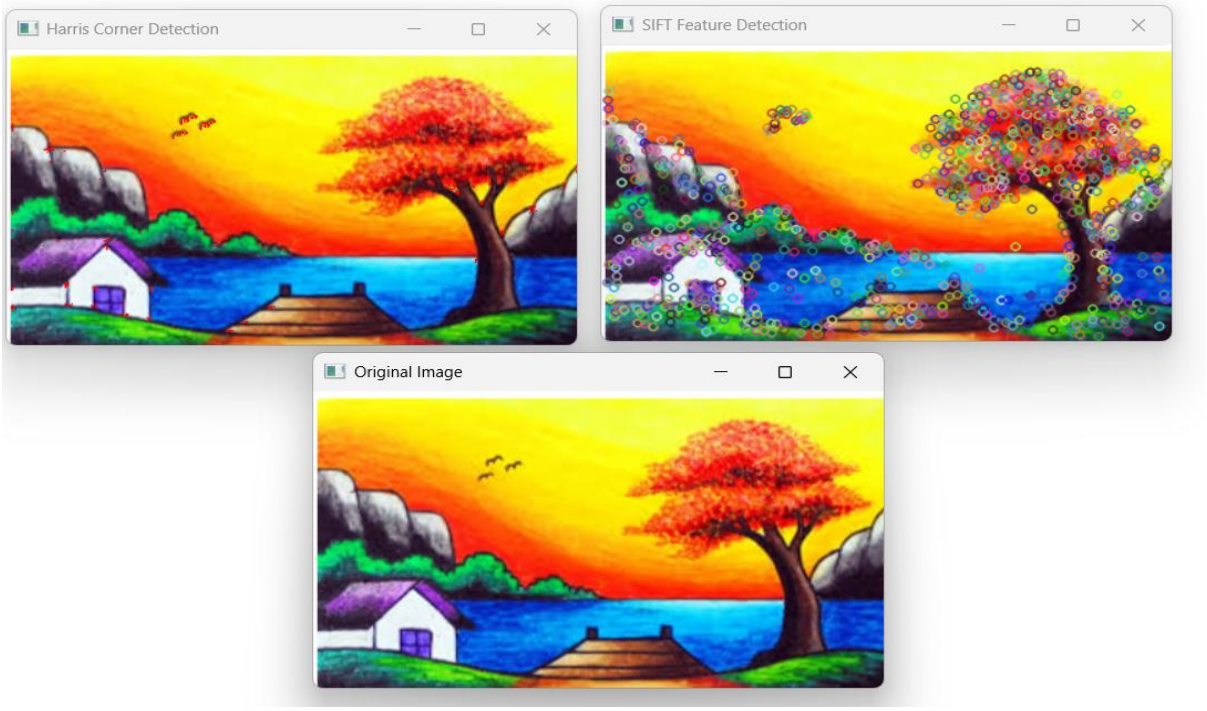
1. Load the image.
2. Convert to grayscale.
3. Detect Harris corners.
4. Detect SIFT keypoints.
5. Draw detected features.
6. Compare the outputs.

Python Code

```
import cv2
import numpy as np
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Harris Corner Detection
gray_float = np.float32(gray)
harris = cv2.cornerHarris(gray_float, 2, 3, 0.04)
img_harris = img.copy()
img_harris[harris > 0.01 * harris.max()] = [0, 0, 255]
# SIFT Detection
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
img_sift = cv2.drawKeypoints(img, kp, None)
cv2.imshow("Original", img)
cv2.imshow("Harris Corners", img_harris)
```

```
cv2.imshow("SIFT Features", img_sift)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

Harris detects only corner points, while SIFT detects more stable and descriptive feature points suitable for image matching.

2.Image Enhancement for Feature Detection

Conduct an experiment to evaluate the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance. Explain the importance of image enhancement in computer vision. Apply enhancement methods followed by feature detection using appropriate tools, document the intermediate and final outputs clearly, and analyze how enhancement influences feature visibility, detection accuracy, and reliability.

A) Aim

To study the effect of image enhancement on feature detection.

Algorithm

1. Read the image.
2. Convert to grayscale.
3. Apply Histogram Equalization.
4. Detect features.
5. Compare before and after enhancement.

Procedure

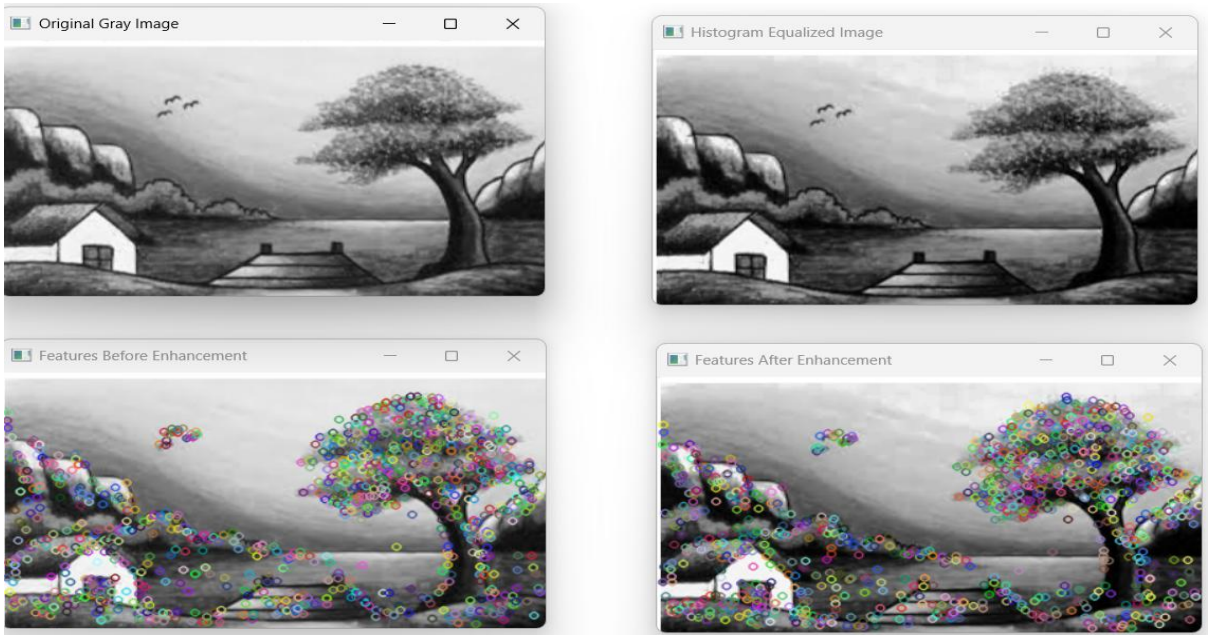
1. Load image.
2. Convert to grayscale.

3. Enhance image using Histogram Equalization.
4. Detect SIFT features.
5. Compare outputs.

Python Code

```
import cv2
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
equalized = cv2.equalizeHist(gray)
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray, None)
kp2, des2 = sift.detectAndCompute(equalized, None)
img1 = cv2.drawKeypoints(gray, kp1, None)
img2 = cv2.drawKeypoints(equalized, kp2, None)
cv2.imshow("Original", gray)
cv2.imshow("Histogram Equalization", equalized)
cv2.imshow("Features Before", img1)
cv2.imshow("Features After", img2)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

Image enhancement improves image contrast, making more feature points visible and increasing detection accuracy.

3.Multi-Scale Feature Detection Analysis

Design and implement an experiment to study feature detection at different image scales. Explain the concept of image scaling and scale-space representation in feature extraction.

Apply feature detection methods on images of varying resolutions using suitable software/tools, document the detected features systematically, and analyze the effect of scale variations on feature stability, accuracy, and computational performance.

A) Aim

To analyze feature detection at different image scales.

Algorithm

1. Read the image.
2. Resize to different scales.
3. Detect SIFT features.
4. Compare detected features.

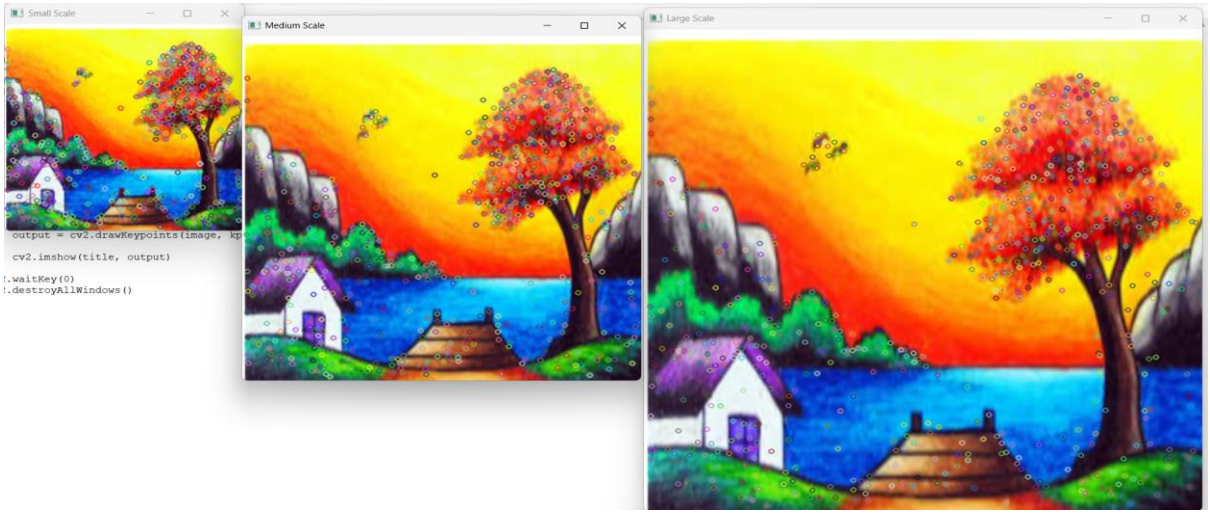
Procedure

1. Load image.
2. Create different image sizes.
3. Detect SIFT features.
4. Display outputs.

Python Code

```
import cv2
img = cv2.imread("image.jpg")
small = cv2.resize(img, (300,300))
medium = cv2.resize(img, (500,500))
large = cv2.resize(img, (700,700))
sift = cv2.SIFT_create()
for title, image in [("Small", small), ("Medium", medium), ("Large", large)]:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    kp, des = sift.detectAndCompute(gray, None)
    out = cv2.drawKeypoints(image, kp, None)
    cv2.imshow(title, out)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

Larger images usually produce more feature points, while SIFT remains robust across different scales.

4.Comparative Analysis of Image Filtering Techniques

Conduct an experiment to compare different image filtering techniques, such as Mean, Gaussian, and Median filtering, for preprocessing. Explain the theoretical concepts and objectives of each filtering method. Implement the filters using appropriate tools/software, document the processed images and observations clearly, and analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

A) Aim

To compare Mean, Gaussian, and Median filters.

Algorithm

1. Read image.
2. Apply Mean Filter.
3. Apply Gaussian Filter.
4. Apply Median Filter.
5. Compare outputs.

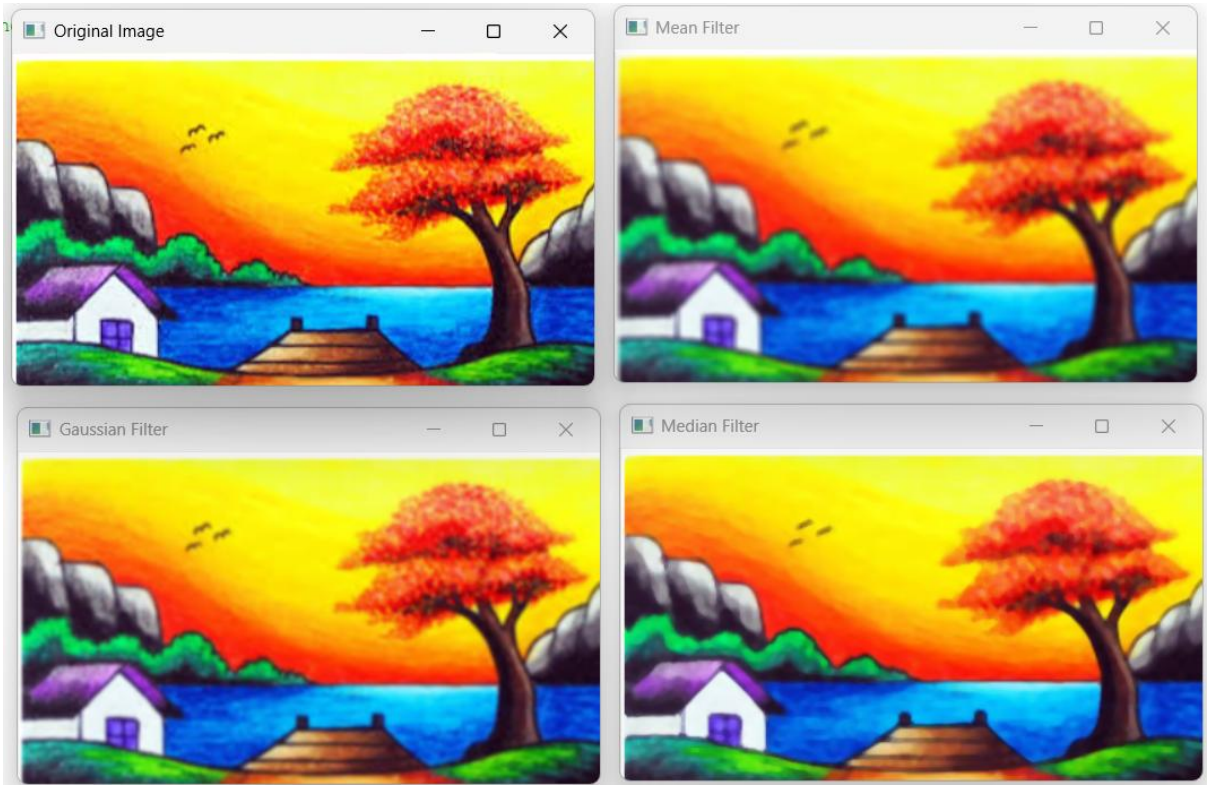
Procedure

1. Load image.
2. Apply all three filters.
3. Display results.
4. Compare image quality.

Python Code

```
import cv2
img = cv2.
imread("image.jpg")
mean = cv2.blur(img, (5,5))
gaussian = cv2.
Gaussian
Blur(img, (5,5), 0)
median = cv2.
Median
Blur(img, 5)
cv2.imshow("Original", img)
cv2.imshow("Mean Filter", mean)
cv2.imshow("Gaussian Filter", gaussian)
cv2.imshow("Median Filter", median)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

Mean filter smooths the image, Gaussian filter reduces Gaussian noise while preserving details better, and Median filter effectively removes salt-and-pepper noise.

5.Complete Object Detection Workflow

Design and perform an experiment to develop an object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection. Explain the purpose and interaction of each processing stage in the workflow. Implement the complete pipeline using suitable software/tools, document the outputs at every stage systematically, and analyze the effectiveness of the integrated approach in achieving accurate and reliable object detection under different image conditions.

A)

Aim

To develop a complete object detection workflow using preprocessing, edge detection, feature extraction, and contour detection.

Algorithm

1. Read image.
2. Convert to grayscale.
3. Apply Gaussian Blur.
4. Detect edges using Canny.
5. Detect SIFT features.
6. Detect contours.
7. Display outputs.

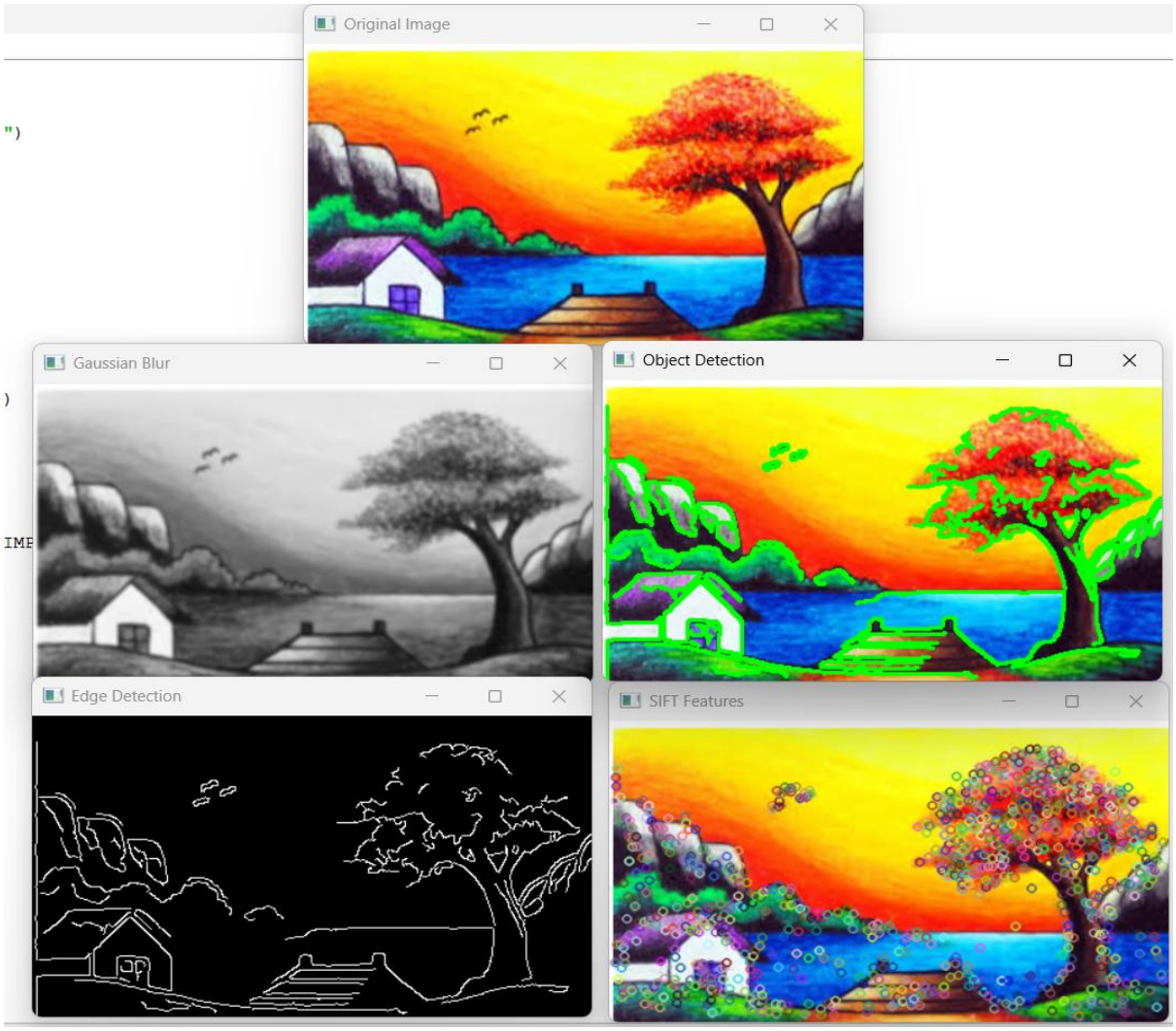
Procedure

1. Load image.
2. Convert to grayscale.
3. Remove noise.
4. Detect edges.
5. Detect SIFT features.
6. Detect object boundaries.
7. Display all outputs.

Python Code

```
import cv2
img = cv2.imread("image.jpg")
gray = cv2.cvt
Color(img, cv2.COLOR_BGR2GRAY)
blur = cv2.Gaussian
Blur(gray, (5,5), 0)
edges = cv2.
Canny(blur, 100, 200)
sift = cv2.
SIFT_create()
kp, des = sift.
detectAnd
Compute
(gray, None)
feature_img = cv2.draw
Keypoints
(img, kp, None)
contours,
hierarchy = cv2.find
Contours(edges,
cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
contour_img = img.copy()
cv2.drawContours
(contour_img, contours, -1, (0,255,0), 2)
cv2.imshow("Original", img)
cv2.imshow("Blur", blur)
cv2.imshow("Edges", edges)
cv2.imshow("SIFT Features", feature_img)
cv2.imshow("Detected Objects", contour_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

The complete workflow successfully preprocesses the image, detects edges, extracts feature points, and identifies object boundaries, providing accurate and reliable object detection.